

UD BMEG 671 Module 1 — Compartmental Modeling Foundations

Based on BMEG/ELEG 471/671 lectures

DiStefano and current modeling guidance

Notes taken by NanFangHong

Module 1 2026

Reading motive

这是 UD BMEG/ELEG 471/671 前三次课的学习笔记。课程标题叫 Advanced Biomedical and Pharmaceutical Modeling，但前三节课真正搭的是一门语言：怎样把生物医学故事翻译成 compartments, states, inputs, outputs, rates, ODEs, code, and validation。我的目标不是逐页复述课件，而是把课堂 transcript、带标注和无标注 lecture notes、*MATLAB* live scripts、DiStefano 的 dynamic systems biology 视角、以及 2026 年仍在更新的 *MIDD*/model credibility 语境接成一个能自学的开端。

[course note]

Reader

假设读者已经见过一点微积分和线性代数，但还没有真正把 ODE、系统、药物动力学、疾病动力学、代码验证和数据拟合放在同一个脑图里。这里会慢一点，把每个符号先问清楚：它是 amount, concentration, rate constant, input, output, measurement, decision variable, or hidden biological state?

[from zero]

Update

课程材料来自 Fall 2025。本笔记保留 Islam/Zurakowski 的课堂主线，同时补入 FDA *MIDD* paired meeting program, ICH M15, ASME V&V 40, *SBML*, and *SimBiology* 的当前语境：模型不只是 homework equation，而是可能进入 regulatory reasoning、model exchange、credibility assessment 和 therapeutic decision 的科学对象。

[2026 view]

Contents

1	The Course Spine	4
1.1	What the first three lectures are really doing	4
1.2	Why this matters in biomedical and pharmaceutical modeling . .	4
1.3	The DiStefano perspective	5
2	The Compartmental Grammar	6
2.1	Compartment, species, state, and output	6
2.2	Same compartment, different species	6
2.3	Different compartments, same species	7
2.4	Central and peripheral lumping	7
3	From Diagram to ODE	8
3.1	The mass-balance sentence	8
3.2	Sign checking	8
3.3	State-space form	8
4	Inputs: Step Versus Impulse	9
4.1	Why inputs deserve their own section	9
4.2	Bolus as impulse	9
4.3	Infusion or production as step	9
4.4	The experimental interpretation	9
5	One Compartment Impulse	11
5.1	Equation and solution	11
5.2	What the solution means	11
5.3	Code pattern	11
5.4	Verification and validation	12
6	One Compartment Step	13
6.1	Equation and solution	13
6.2	Why different initial values converge	13
6.3	Drug interpretation	13
6.4	Step-input code pattern	13
7	The MATLAB Pattern	15
7.1	Why code appears so early	15
7.2	ODE function shape	15
7.3	Parameter passing and reusable code	15
7.4	Impulse versus step in code	16
7.5	ode45 and ode15s	16
7.6	deval versus array output	16
8	Two Compartment Linear Model	17
8.1	Diagram	17
8.2	Equations	17
8.3	Two-compartment code as a sign audit	17
8.4	Steady state for the in-class parameters	18
8.5	What reversing the input teaches	18
8.6	Output equations	18

9	The Butyrate Translation Exercise	19
9.1	Why this example matters	19
9.2	A minimal two-state version	19
9.3	Adding a bone state	19
9.4	The key modeling lesson	19
10	Nonlinear Elimination	20
10.1	Why linearity breaks	20
10.2	Steady state condition	20
10.3	Code scan for capacity stress testing	20
10.4	Why this matters pharmacologically	21
10.5	Local linearization	21
11	The SIR Model as a Compartmental Model	22
11.1	Equations	22
11.2	SIR code pattern	22
11.3	Conservation	22
11.4	Units	23
11.5	Why SIR belongs in this course	23
12	Verification, Validation, and Credibility	24
12.1	A practical distinction	24
12.2	ASME V and V 40	24
12.3	MIDD and the question of interest	24
12.4	Model exchange and SBML	24
13	Choosing a Course Project	25
13.1	The assignment logic	25
13.2	Good project shapes	25
13.3	Warning signs	25
14	A Study Checklist for the First Three Lectures	26
14.1	Before writing equations	26
14.2	While writing equations	26
14.3	After coding	26
14.4	Before believing the model	27
15	Worked Micro-Examples	28
15.1	Example 1: a urinary excretion accounting state	28
15.2	Example 2: a reversible distribution state	28
15.3	Example 3: a simple effect compartment	28
16	Summary	29
16.1	One paragraph	29
16.2	The sentence to keep	29

1 The Course Spine

1.1 What the first three lectures are really doing

The syllabus says the course will cover compartmental network diagrams, linear systems theory, controllability, observability, reachability, detectability, non-linear interactions, stability, identifiability, sensitivity analysis, likelihood functions, parameter estimation, uncertainty, Markov chain Monte Carlo, model selection, and statistical evaluation of fitted models [13]. That sounds like many topics. The first three lectures reveal the hidden spine:

diagram $\longrightarrow$ mass balance $\longrightarrow$ ODE
 $\longrightarrow$ simulation $\longrightarrow$ data comparison $\longrightarrow$ decision.

This is the same grammar that later appears under many names:

- in pharmacokinetics: dose $\rightarrow$ concentration $\rightarrow$ exposure $\rightarrow$ response;
- in systems biology: reaction network $\rightarrow$ state equations $\rightarrow$ pathway dynamics;
- in control theory: state $\rightarrow$ input $\rightarrow$ output $\rightarrow$ observability and controllability;
- in inference: latent state $\rightarrow$ measurement model $\rightarrow$ likelihood $\rightarrow$ uncertainty;
- in model credibility: question of interest $\rightarrow$ model use $\rightarrow$ verification and validation evidence.

So the first three lectures are not merely “how to write a one-compartment ODE.” They are a compact version of the whole course: choose a state, write a conservation law, solve or simulate, verify the implementation, validate against a truth source, and then ask whether the model is fit for its intended use.

1.2 Why this matters in biomedical and pharmaceutical modeling

Biomedical systems are often observable only through partial, delayed, noisy measurements. We may dose the blood, observe plasma concentration, care about a tissue target, and make a clinical decision about efficacy or toxicity. The thing we want is rarely the thing we directly measure.

Compartmental modeling is a disciplined compromise. It says: do not try to draw every cell. Instead, choose a small number of states whose changes explain the data and decision. Each state is a storage variable: amount of drug in blood, amount of metabolite in urine, number of infected individuals, abundance of a signaling species, mass of butyrate in gut, or active osteoblast population.

The course begins here because every later method depends on this first act of abstraction. If the state variables are ill-chosen, no algorithm can rescue the model. If the units are inconsistent, the simulation may be beautiful but meaningless. If the output equation does not correspond to the measurement, parameter estimation will learn the wrong thing.

1.3 The DiStefano perspective

DiStefano's textbook frames dynamic systems biology as modeling and simulation of systems whose variables change in time [2]. The key word is dynamic. A static regression might say that high concentration correlates with high effect. A dynamic model asks how concentration becomes high, when it becomes high, why it decays, and what hidden rates control that time course.

For this course, the useful mental model is:

$$\begin{array}{ccc}
 \boxed{\text{biological story}} & \xrightarrow{\text{state choice}} & \boxed{\dot{\mathbf{q}} = f(\mathbf{q}, \mathbf{u}, \boldsymbol{\theta}, t)} \\
 & \xrightarrow{\text{output map}} & \boxed{\mathbf{y} = g(\mathbf{q}, \boldsymbol{\theta}, t)}.
 \end{array}$$

The story is qualitative. The ODE is quantitative. The output map is what makes it testable.

2 The Compartmental Grammar

2.1 Compartment, species, state, and output

A compartment is not merely a box in a diagram. In Lecture 1, the working definition is a physical or conceptual space in which the quantity of interest is treated as homogeneously distributed, with mixing inside the compartment faster than transfer between compartments [4, 6]. The word “treated” matters. A compartment is a modeling decision, not a claim that nature literally contains boxes.

We need four related but distinct objects:

- A *compartment* is the location or pool: blood, urine, gut, bone, susceptible population, infected population.
- A *species* is what is counted: parent drug, metabolite, T4, T3, butyrate, infected individuals.
- A *state variable* is the time-varying amount or concentration represented in the model: $q(t)$, $C(t)$, $S(t)$, $I(t)$, $R(t)$.
- An *output* is what the experiment reports: a plasma concentration, urinary amount, biomarker, imaging measurement, viral load, or clinical score.

The trap is to collapse all four into one word. In a simple drug model, blood may be the compartment, parent drug the species, $q_{\text{blood}}(t)$ the state, and measured plasma concentration $C_p(t) = q_{\text{blood}}(t)/V_p$ the output. If the experiment measures urine, the output is not the central amount; it is a different state or a derived cumulative excretion.

2.2 Same compartment, different species

The transcript uses the thyroid example to show that a compartment need not correspond one-to-one to a state. T4 and T3 can both exist inside a cell. The physical space is the same, but the species differ, so the model needs two state variables:

$$q_{\text{T4,cell}}(t), \quad q_{\text{T3,cell}}(t).$$

If T4 is converted to T3, the arrow is not a movement between physical rooms. It is a reaction between species within a location. In a diagram, both movement and reaction may look like arrows, but their interpretation differs.



This distinction becomes essential in systems biology and *QSP* because compartments often contain many species. A liver compartment might contain parent drug, metabolite, enzyme, transporter, receptor, and signal. The same physical organ can generate a high-dimensional state vector.

2.3 Different compartments, same species

The blood/urine example is the opposite. The species may be the same drug, but the locations differ:

$$q_{\text{blood}}(t), \quad q_{\text{urine}}(t).$$

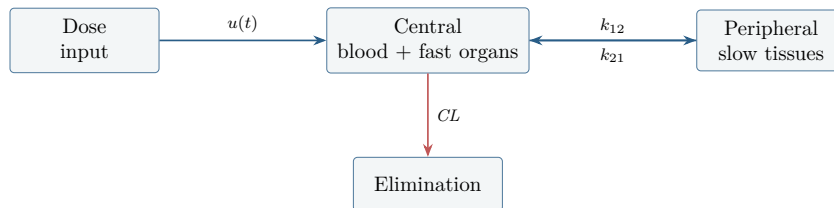
If urine amount is cumulative excretion, then the urine state may have no out-flow over the sampling window:

$$\frac{d}{dt}q_{\text{urine}}(t) = k_{\text{renal}}q_{\text{blood}}(t).$$

This state is not a storage compartment with reversible exchange; it is an accounting compartment. That is perfectly legitimate, but it changes the interpretation of the arrow.

2.4 Central and peripheral lumping

The transcript also gives a pharmacokinetic example: blood and highly perfused organs may be lumped into a central compartment, while slower tissues become peripheral compartments [6]. The point is not that liver, kidney, lung, brain, and blood are physically one place. The point is that, at the time scale and measurement resolution of the experiment, they may equilibrate quickly relative to other transfers.



Lumping is powerful, but it is also dangerous. It hides spatial detail. A central compartment volume is not a literal anatomical volume; it is an apparent volume compatible with the observed concentration-time curve. A useful rule is:

lump states only when the lost detail is not needed for the question of interest.

This phrase anticipates ICH M15: model quality is judged relative to the question the model is meant to answer, not in the abstract [3].

3 From Diagram to ODE

3.1 The mass-balance sentence

The first mathematical sentence of the course is:

rate of change inside = rate in – rate out + rate produced – rate consumed.

For a single amount $q(t)$, this becomes

$$\frac{d}{dt}q(t) = \sum \text{inflows} - \sum \text{outflows}.$$

Everything else is elaboration.

If a first-order loss has rate constant k , the outflow is $kq(t)$. This is not magic. The units force it:

$$[k] = \text{time}^{-1}, \quad [q] = \text{amount}, \quad [kq] = \text{amount/time}.$$

In the transcript, the instructor emphasizes a subtle point: dimension is not enough; units must match as well [7]. A term in kg/s cannot be added directly to kg/hour until it is converted.

3.2 Sign checking

Every arrow leaving a state should appear with a minus sign in that state's equation. Every arrow entering a state should appear with a plus sign. If a two-compartment diagram has transfer $k_{21}q_1$ from state 1 to state 2, then:

$$\dot{q}_1 \text{ contains } -k_{21}q_1, \quad \dot{q}_2 \text{ contains } +k_{21}q_1.$$

This is the simplest verification test. Before running *MATLAB*, check whether each arrow appears twice, once as a loss and once as a gain, unless it is an external input or external sink.

3.3 State-space form

Linear compartmental models can be written as

$$\dot{\mathbf{q}}(t) = \mathbf{A}\mathbf{q}(t) + \mathbf{B}\mathbf{u}(t), \quad \mathbf{y}(t) = \mathbf{C}\mathbf{q}(t).$$

Here $\mathbf{q}$ is the state vector, $\mathbf{u}$ is the input vector, and $\mathbf{y}$ is the output vector. This notation is not merely cosmetic. It is the bridge to the later course topics:

- controllability asks whether inputs can move the state through relevant directions;
- observability asks whether outputs contain enough information to reconstruct states;
- identifiability asks whether parameters can be inferred from outputs;
- stability asks whether trajectories decay, grow, or settle;
- sensitivity asks which parameter perturbations change outputs.

The first three lectures only touch the edge of these topics, but the notation is already present.

4 Inputs: Step Versus Impulse

4.1 Why inputs deserve their own section

Inputs are where mathematical idealization meets experimental protocol. An intravenous bolus, a constant infusion, an oral dose, an endogenous production rate, and a viral exposure are all “inputs,” but they enter an ODE differently.

The lectures introduce two ideal inputs:

$$\mathbf{1}(t) = \begin{cases} 0, & t < 0, \\ 1, & t \geq 0, \end{cases} \quad \delta(t).$$

The unit step turns something on and keeps it on. The Dirac impulse deposits a finite amount at an instant.

4.2 Bolus as impulse

The one-compartment bolus model is written as

$$\frac{d}{dt}q(t) = Q_0\delta(t) - kq(t).$$

The impulse term means that a total amount Q_0 is placed into the system at $t = 0$. For $t > 0$, the equation is simply

$$\frac{d}{dt}q(t) = -kq(t), \quad q(0^+) = Q_0.$$

This is why the *MATLAB* live script treats the impulse as an initial condition rather than trying to evaluate a Dirac delta numerically [10, 7].

4.3 Infusion or production as step

A constant infusion or endogenous production rate is different:

$$\frac{d}{dt}q(t) = IR\mathbf{1}(t) - kq(t).$$

For $t \geq 0$, this is

$$\frac{d}{dt}q(t) = IR - kq(t).$$

The input persists at every time. In code, it belongs inside the right-hand-side function, not just in the initial condition.

4.4 The experimental interpretation

The same equation can describe different biomedical protocols:

- $Q_0\delta(t)$: IV bolus drug dose, instantaneous tracer injection, one-time inoculum.
- $IR\mathbf{1}(t)$: constant IV infusion, continuous nutrient supply, zero-order endogenous production, step change in stimulus.

- $u(t)$: arbitrary time-varying dose schedule, pulsatile hormone input, piece-wise dosing regimen.

The modeler should always write the protocol in words before choosing $u(t)$. A common beginner error is to choose an input because the equation is familiar, not because it matches the experiment.

5 One Compartment Impulse

5.1 Equation and solution

For an impulse input at $t = 0$, the post-dose model is

$$\frac{d}{dt}q(t) = -kq(t), \quad q(0) = Q_0.$$

Separation of variables gives

$$q(t) = Q_0 e^{-kt}.$$

The concentration version is obtained by dividing by a volume:

$$C(t) = \frac{q(t)}{V} = C_0 e^{-kt}.$$

The half-life is

$$t_{1/2} = \frac{\log 2}{k}.$$

In pharmacokinetics, if $k = CL/V_d$, then

$$t_{1/2} = \frac{\log 2 V_d}{CL}.$$

This formula is a useful preview of later *PK* notes: half-life is not a primitive mechanism. It is a derived consequence of clearance and volume.

5.2 What the solution means

The exponential curve says that the amount lost per unit time is proportional to the amount present. At high amount, loss is fast. At low amount, loss is slow. The system has no finite positive steady state because there is no continuing input:

$$q(t) \rightarrow 0.$$

The slope on a semilog plot is $-k$. This is why first-order elimination is visually recognizable. If data bend away from a straight semilog line, the one-compartment first-order model may be missing absorption, distribution, nonlinear elimination, assay issues, or multiple phases.

5.3 Code pattern

The live script's code pattern is conceptually:

```
t = 0:0.1:10;
k = 1;
Q0 = 1;
qi = Q0;
soln = ode45(@(t,y) ODEs(t,y), t, qi);
q_sol = deval(soln,t,1);
true_sol = Q0*exp(-k*t);
```

and the right-hand side is:

```
function derivVector = ODEs(t,y)
    k = 1;
    q = y(1);
    dqdt = -k*q;
    derivVector = [dqdt]';
end
```

This code teaches three habits:

- use an ODE function that takes t and y , even if t is not explicitly used;
- return a column vector, hence the transpose in `[dqdt]'`;
- compare with an analytical solution when one exists.

5.4 Verification and validation

The lecture distinguishes verification from validation [7]. For this example:

- Verification: did the code implement $\dot{q} = -kq$ correctly? Does the numerical solution match $Q_0 e^{-kt}$?
- Validation: does this equation describe a real experimental concentration-time curve well enough for the intended question?

Analytical comparison verifies numerical implementation. It does not automatically validate biological adequacy.

6 One Compartment Step

6.1 Equation and solution

For a constant input after time zero:

$$\frac{d}{dt}q(t) = IR - kq(t), \quad q(0) = q_0.$$

The solution is

$$q(t) = \frac{IR}{k} + \left(q_0 - \frac{IR}{k}\right)e^{-kt}.$$

The steady state is

$$q^* = \frac{IR}{k}.$$

This is one of the most important formulas in the first week. It says:

$$\text{steady state amount} = \frac{\text{input rate}}{\text{fractional loss rate}}.$$

6.2 Why different initial values converge

The step live script compares two initial values and shows both trajectories approach the same steady state [11]. That happens because the system is linear and stable. Initial condition controls transient memory; input and loss rate control the eventual equilibrium.

If $q_0 < IR/k$, the curve rises. If $q_0 > IR/k$, the curve falls. If $q_0 = IR/k$, the system is already at steady state. The exponential factor e^{-kt} is the memory of the initial condition.

6.3 Drug interpretation

For an IV infusion with central amount q , concentration $C = q/V$, and elimination rate constant $k = CL/V$:

$$C^* = \frac{IR}{CL}.$$

This is the familiar pharmacokinetic steady-state relationship. Again, the course's simple ODE is already a real drug-development tool: it tells us how a maintenance infusion rate maps to target concentration.

6.4 Step-input code pattern

The corresponding live script should be read as a miniature experiment. The input rate is not hidden in the initial condition; it lives inside the right-hand side:

```
t = 0:0.1:10;
k = 1;
IR = 1;
qi = 0;
soln = ode45(@(t,y) ODEs(t,y), t, qi);
q_sol = deval(soln,t,1);
q_ss = IR/k;
```

with

```
function derivVector = ODEs(t,y)
    k = 1;
    IR = 1;
    q = y(1);
    dqdt = IR - k*q;
    derivVector = [dqdt]';
end
```

This tiny difference from the impulse script is the whole modeling lesson. An impulse dose changes $q(0)$. A sustained infusion changes $\dot{q}$ for as long as the infusion is active. If the code and the intended experiment disagree here, every later plot may look smooth and still be answering the wrong question.

7 The MATLAB Pattern

7.1 Why code appears so early

The lectures put *MATLAB* in the first week because modeling is not only equation writing. A model becomes operational only when it can be simulated, checked, and compared. In *QSP*, *PBPK*, and *PK/PD* workflows, the gap between a diagram and a credible result often lives in code details: units, signs, solver choice, parameter passing, initial conditions, and output extraction.

MathWorks' *SimBiology* tutorials now explicitly frame the tool around *QSP*, *PBPK*, and *PK/PD* modeling and analysis [14]. This is exactly why the course asks students to become comfortable with programmatic ODE simulation before using higher-level graphical tools.

7.2 ODE function shape

The canonical shape is:

```
function derivVector = ODEs(t,y)
    q1 = y(1);
    q2 = y(2);
    dq1dt = ...;
    dq2dt = ...;
    derivVector = [dq1dt,dq2dt]';
end
```

The solver does not care what the states mean. It receives a vector and returns a derivative vector. The meaning is carried by our naming, equations, and documentation.

7.3 Parameter passing and reusable code

The first-week scripts deliberately hard-code simple parameters inside *ODEs*. That makes the classroom algebra easy to see. As soon as the same model is scanned over doses, rates, or patient values, the safer habit is to pass parameters through a closure:

```
theta = [k, IR];
odeFunc = @(t,y) ODEs(t,y,theta);
soln = ode45(odeFunc, t, qi);

function derivVector = ODEs(t,y,theta)
    k = theta(1);
    IR = theta(2);
    q = y(1);
    dqdt = IR - k*q;
    derivVector = [dqdt]';
end
```

This is not software decoration. It separates model structure from the experiment being run. Later vaccine and stability scripts use exactly this idea: the ODE function encodes mechanism, while the outer script changes inputs, parameters, initial conditions, or time windows.

7.4 Impulse versus step in code

The transcript's distinction is worth making explicit:

impulse dose	⇒	put dose in initial condition,
step input	⇒	put rate inside ODE function.

If an impulse is coded as a finite input at every time step, the simulation adds too much mass. If a continuous infusion is coded only as an initial condition, the input stops immediately. These are not small coding choices; they change the experiment being simulated.

7.5 ode45 and ode15s

The transcript contrasts nonstiff and stiff solvers. In ordinary *MATLAB* usage, `ode45` is a default nonstiff Runge–Kutta solver, while `ode15s` is commonly used for stiff systems [7, 8]. Stiffness often appears when a model has both very fast and very slow processes. Biomedical models frequently do: receptor binding may equilibrate rapidly, while disease progression evolves over weeks.

The beginner lesson is not to memorize solver taxonomy. It is to watch for solver warnings, step-size pathologies, impossible negative concentrations, and sensitivity to tolerances.

7.6 deval versus array output

The first live scripts use `deval` to evaluate the solution at requested time points. The SIR-style example discussed in transcript also uses the direct array output form from the solver. Both are legitimate. The deeper point is to keep the time vector and output columns aligned:

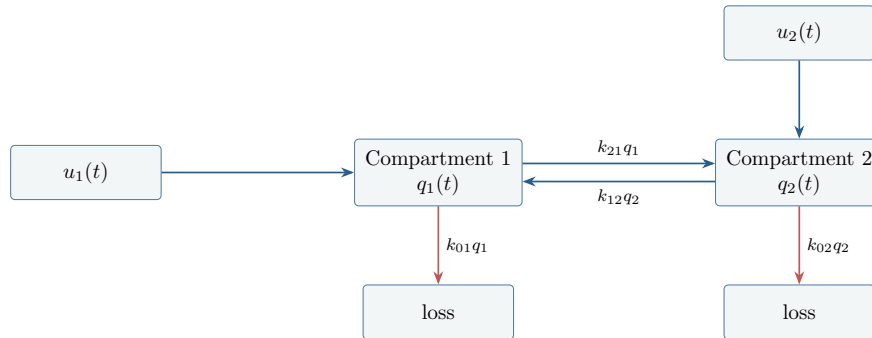
column 1 = state 1, column 2 = state 2, ...

Many modeling errors are indexing errors wearing scientific clothing.

8 Two Compartment Linear Model

8.1 Diagram

Lecture 1 introduces a two-compartment model; Lecture 3 turns it into *MATLAB* code [4, 5, 12]. Let q_1 and q_2 be amounts in two compartments. Let $u_1(t)$ and $u_2(t)$ be external inputs. Let k_{01} and k_{02} be losses to the outside. Let $k_{21}q_1$ be transfer from 1 to 2, and $k_{12}q_2$ transfer from 2 to 1.



8.2 Equations

Mass balance gives

$$\begin{aligned}\frac{d}{dt}q_1 &= -(k_{01} + k_{21})q_1 + k_{12}q_2 + u_1(t), \\ \frac{d}{dt}q_2 &= k_{21}q_1 - (k_{02} + k_{12})q_2 + u_2(t).\end{aligned}$$

In matrix form:

$$\frac{d}{dt} \begin{bmatrix} q_1 \\ q_2 \end{bmatrix} = \underbrace{\begin{bmatrix} -(k_{01} + k_{21}) & k_{12} \\ k_{21} & -(k_{02} + k_{12}) \end{bmatrix}}_{\mathbf{A}} \begin{bmatrix} q_1 \\ q_2 \end{bmatrix} + \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}.$$

Notice the diagonal terms are negative sums of all first-order exits from that compartment. The off-diagonal terms are gains from the other compartment. This pattern generalizes to large compartmental systems.

8.3 Two-compartment code as a sign audit

The two-compartment live script is especially useful because each code line can be checked against one arrow in the diagram:

```
q1 = y(1);
q2 = y(2);
dq1dt = -(k01+k21)*q1 + k12*q2 + IR1;
dq2dt = k21*q1 - (k02+k12)*q2 + IR2;
derivVector = [dq1dt,dq2dt]';
```

The diagonal loss terms $-(k_{01} + k_{21})q_1$ and $-(k_{02} + k_{12})q_2$ are the easiest place to make a sign error. The transfer $k_{21}q_1$ must appear once as a loss from

compartment 1 and once as a gain to compartment 2. The transfer $k_{12}q_2$ must do the opposite. In practice I would mark the diagram and code with the same colors before trusting a simulation.

The script's input reversal exercise is also a code-level identifiability lesson. Changing IR1 and IR2 changes which state receives external mass. If symmetric parameters and symmetric inputs make the curves overlap, the overlap is not a numerical accident; it is a structural property of the model.

8.4 Steady state for the in-class parameters

The live script uses

$$k_{01} = k_{02} = k_{12} = k_{21} = 1, \quad u_1 = 2, \quad u_2 = 0.$$

At steady state:

$$\begin{aligned} 0 &= -2q_1 + q_2 + 2, \\ 0 &= q_1 - 2q_2. \end{aligned}$$

The second equation gives $q_1 = 2q_2$. Substitution gives

$$-4q_2 + q_2 + 2 = 0, \quad q_2^* = \frac{2}{3}, \quad q_1^* = \frac{4}{3}.$$

This is a useful check against the numerical simulation. If the plotted curves do not settle near these values, something is wrong in the code, parameter assignment, or interpretation of input direction.

8.5 What reversing the input teaches

The transcript notes that reversing the inputs reverses the curves, and equal inputs can make the two curves overlap under symmetric rates [8]. That observation is a gentle introduction to symmetry and identifiability. If two compartments have symmetric rates and the same input-output structure, they may become hard to distinguish from data. Model structure determines what can be learned.

8.6 Output equations

The two-compartment state is not automatically the measured data. If only compartment 1 is measured:

$$y(t) = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} q_1(t) \\ q_2(t) \end{bmatrix}.$$

If concentration is measured, not amount:

$$y(t) = \frac{q_1(t)}{V_1}.$$

If assay noise is present:

$$\tilde{y}_i = y(t_i) + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma^2).$$

This small output layer is the doorway from deterministic simulation to statistical inference.

9 The Butyrate Translation Exercise

9.1 Why this example matters

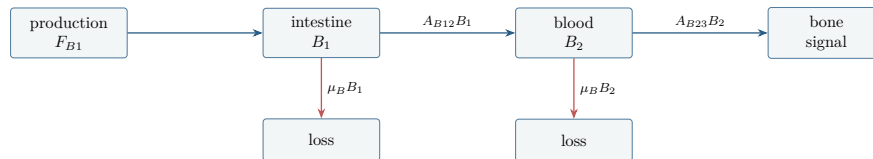
The lectures use a butyrate and bone-formation example drawn from the instructor's research area. Biologically, butyrate is a short-chain fatty acid produced by gut microbiota; experimental work such as Tyagi et al. links probiotic-induced butyrate increases to regulatory T cell expansion, WNT10B signaling, and bone formation [16]. The classroom task is not to reproduce the full biology. It is to practice translation:

verbal pathway $\longrightarrow$ compartment diagram $\longrightarrow$ ODEs.

9.2 A minimal two-state version

Let $B_1(t)$ be butyrate amount in intestine and $B_2(t)$ be butyrate amount in blood. Suppose butyrate is formed in intestine at rate F_{B1} , exits intestine and blood with first-order rate μ_B , transfers from intestine to blood with rate constant A_{B12} , and transfers from blood toward bone with rate constant A_{B23} . If bone is treated as a downstream sink rather than a state, a minimal model is:

$$\begin{aligned}\frac{d}{dt}B_1 &= F_{B1} - (\mu_B + A_{B12})B_1, \\ \frac{d}{dt}B_2 &= A_{B12}B_1 - (\mu_B + A_{B23})B_2.\end{aligned}$$



9.3 Adding a bone state

If bone exposure or effect is explicitly modeled, add a third state:

$$\frac{d}{dt}B_{\text{bone}} = A_{B23}B_2 - k_{\text{bone}}B_{\text{bone}}.$$

This is a modeling choice. If the data include only gut and serum butyrate, a bone state may be weakly identifiable. If the data include bone biomarkers or bone formation rate, the bone state may be needed. This is exactly the kind of judgment the course wants students to develop.

9.4 The key modeling lesson

The butyrate example has a richer biological story than the one-compartment drug model, but the mathematical grammar is the same:

formation $-$ loss $-$ transfer out $+ \text{transfer in}$.

Once this grammar is internalized, many biomedical systems become approachable.

10 Nonlinear Elimination

10.1 Why linearity breaks

Lecture 3 introduces a nonlinear one-compartment model:

$$\frac{d}{dt}q(t) = IR - \frac{Cq(t)}{K + q(t)}.$$

Here K has units of amount and controls the saturation scale; C has units of amount/time and controls the maximum elimination capacity [8]. This is Michaelis–Menten-like elimination written in amount units.

For small q relative to K ,

$$\frac{Cq}{K + q} \approx \frac{C}{K}q.$$

So the system behaves approximately like a linear first-order elimination model with $k = C/K$. For large q ,

$$\frac{Cq}{K + q} \approx C.$$

The elimination rate saturates.

10.2 Steady state condition

At steady state:

$$IR = \frac{Cq^*}{K + q^*}.$$

Solving:

$$IR(K + q^*) = Cq^*, \quad IRK = (C - IR)q^*,$$

so

$$q^* = \frac{IRK}{C - IR}.$$

This exists as a positive finite steady state only when

$$IR < C.$$

If input rate approaches capacity, q^* grows sharply. If input rate exceeds capacity, no finite steady state exists; the amount keeps rising. This is the mathematical version of the lecture's qualitative statement that high input can overwhelm elimination.

10.3 Code scan for capacity stress testing

The nonlinear live script turns the algebra into a stress test by changing the input rate while keeping C and K fixed [9]:

```
IRs = [0.01 0.02 1 2.5 5];
C = 1;
K = 1;
```

```

function derivVector = ODEs(t,y,IR,C,K)
    q = y(1);
    dqdt = IR - C*q/(K+q);
    derivVector = [dqdt]';
end

```

The pedagogical point is that IR s is not just a plotting convenience. It asks whether the qualitative behavior is stable across experimental regimes. For $IR < C$, trajectories can settle. For $IR \geq C$, the code should show continued growth, because the modeled system has no finite positive steady state. If a numerical plot appears to “plateau” in that regime, it is a warning to inspect the time horizon, axis scaling, solver tolerances, or equation implementation.

10.4 Why this matters pharmacologically

Linear models are forgiving: doubling input doubles steady state. Saturable models are not. Near capacity, a small increase in dose can produce a large increase in exposure. This is why nonlinear elimination is not just a mathematical curiosity; it is a safety issue. Classic examples in pharmacokinetics include saturable metabolism, saturable transport, and target-mediated drug disposition.

10.5 Local linearization

Even nonlinear models can be studied locally. Let

$$f(q) = IR - \frac{Cq}{K + q}.$$

Then

$$f'(q) = -\frac{CK}{(K + q)^2}.$$

Near a steady state q^* , perturbations $\Delta q = q - q^*$ approximately satisfy

$$\frac{d}{dt}\Delta q \approx f'(q^*)\Delta q.$$

Because $f'(q^*) < 0$, the steady state is locally stable when it exists. This tiny calculation is the bridge from nonlinear biological models to linear stability analysis later in the course.

11 The SIR Model as a Compartmental Model

11.1 Equations

The SIR model appears in Lecture 3 as another compartmental system:

$$\begin{aligned}\frac{d}{dt}S &= -\beta SI, \\ \frac{d}{dt}I &= \beta SI - aI, \\ \frac{d}{dt}R &= aI.\end{aligned}$$

Here S , I , and R are susceptible, infected, and recovered populations or fractions. The infection term βSI is nonlinear because it depends on contact between susceptible and infected individuals.

11.2 SIR code pattern

The SIR live script uses the same state-vector grammar as the PK scripts:

```
S = y(1);
I = y(2);
R = y(3);

dSdt = -beta*S*I;
dIdt = beta*S*I - a*I;
dRdt = a*I;
derivVector = [dSdt,dIdt,dRdt]';
```

This is where code becomes a mass-balance proof. The term βSI disappears from S and appears in I ; the term aI disappears from I and appears in R . Adding the three code lines should give zero, just as adding the equations gives conservation. A reader can therefore verify the script before thinking about epidemiology.

The code also makes the nonlinear leap visible. In the one-compartment model, every flow was proportional to one state. Here infection is proportional to a product of two states. That product is the doorway to thresholds, local linearization, and stability analysis in Module 2.

11.3 Conservation

Adding the equations gives

$$\frac{d}{dt}(S + I + R) = 0.$$

So $S + I + R$ is conserved. If the initial values are fractions and sum to one, they remain fractions summing to one. This is the same conservation instinct as mass balance, now applied to population categories.

11.4 Units

If S and I are concentrations or population amounts, β must carry the reciprocal units needed to make βSI a rate. If S and I are fractions, units simplify. The transcript notes that biological modeling tools may require biological units rather than purely physical units [8]. This is not pedantry: units determine whether the model can be interpreted, shared, and checked.

11.5 Why SIR belongs in this course

At first, SIR may feel unrelated to drug distribution. But structurally it is the same kind of object:

$$\dot{\mathbf{q}} = f(\mathbf{q}, \boldsymbol{\theta}).$$

The difference is that transfer from S to I depends on two states rather than one. This nonlinear interaction is a preview of immune response, tumor growth, virus-host dynamics, and many disease models.

12 Verification, Validation, and Credibility

12.1 A practical distinction

The first week gives a student-friendly distinction:

- Verification asks: did we implement the intended equations correctly?
- Validation asks: do the intended equations adequately represent the real system for the intended use?

This distinction will become increasingly important as models become too complex for analytical solutions.

For the one-compartment impulse model, the analytical solution $Q_0 e^{-kt}$ is a verification oracle. For a butyrate or SIR model, the truth source may be experimental data, literature ranges, qualitative biological behavior, or independent simulations.

12.2 ASME V and V 40

ASME V&V 40 frames credibility as evidence that a computational model is adequate for a specified context of use, with rigor commensurate with decision consequence [1]. Although V&V 40 was written for medical devices, the logic transfers well to biomedical and pharmaceutical modeling:

more consequential model use $\Rightarrow$ stronger credibility evidence needed.

A homework simulation needs clear equations and code checks. A dose-selection model affecting a clinical program needs documented assumptions, data provenance, sensitivity analysis, uncertainty analysis, external checks, and transparent communication of limitations.

12.3 MIDD and the question of interest

FDA's *MIDD* paired meeting program and ICH M15 both formalize the idea that models can support drug development decisions when the model question, evidence, assumptions, and uncertainty are clear [17, 3]. The phrase "question of interest" is especially useful. A model should not be judged by whether it is complicated. It should be judged by whether it answers a specific question:

Can this model support this decision under this uncertainty?

That question is already visible in the course's HW1 instructions. A chosen paper/model should have real measurements, a manageable ODE state dimension, and a scientifically meaningful comparison between model and data [5, 8].

12.4 Model exchange and SBML

As models grow, reproducibility becomes a real problem. *SBML* is a community standard for representing systems biology models in machine-readable form [15]. Students do not need to master *SBML* in week one, but they should understand the motivation: a model is more credible when its equations, parameters, units, compartments, species, and events are represented in a portable, inspectable form.

13 Choosing a Course Project

13.1 The assignment logic

Lecture 3 explains HW1 as the beginning of a semester-long modeling arc. The student chooses a biomedical or pharmaceutical ODE model, ideally with 3–10 states, real measurements for comparison, and a unique primary article [5, 8]. This is not busywork. It is a way to make every later technique attach to one concrete system.

13.2 Good project shapes

Good early projects often have:

- a clear compartment diagram;
- a small state vector;
- documented parameter values or estimable parameters;
- data over time, not merely endpoint data;
- one or two scientifically meaningful outputs;
- a reason to care about prediction or mechanism.

Examples include a simple *PK/PD* model, a tumor growth inhibition model, a viral dynamics model, an immune response model, a glucose-insulin model, a bone remodeling model, or a reduced *QSP* module.

13.3 Warning signs

Warning signs include:

- a model with dozens of states before the student knows what each state means;
- no available time-course data;
- parameters copied without units;
- a diagram that cannot be translated into equations;
- equations with terms that do not share units;
- a project question that is too broad, such as “model cancer.”

The best project question is narrow enough to simulate and broad enough to matter:

How does changing this rate or input alter this observable time course?

14 A Study Checklist for the First Three Lectures

14.1 Before writing equations

Ask:

- What is the biological or pharmaceutical story?
- What are the compartments?
- What species is tracked in each compartment?
- Are states amounts, concentrations, fractions, or counts?
- Which arrows are transfers, which are reactions, which are external inputs, and which are losses?
- What is measured?

14.2 While writing equations

Check:

- each inflow appears with a plus sign;
- each outflow appears with a minus sign;
- each internal transfer appears in two equations with opposite signs;
- every term in a differential equation has the same units;
- input type matches experiment: impulse, step, or time-varying;
- initial conditions match the protocol.

14.3 After coding

Verify:

- known analytical solutions are reproduced;
- mass conservation holds when it should;
- steady states match algebraic calculations;
- curves have biologically plausible signs and magnitudes;
- solver choices and tolerances do not change conclusions unexpectedly;
- plotted outputs correspond to measured quantities, not merely hidden states.

14.4 Before believing the model

Validate:

- compare predictions with data not used only for coding checks;
- inspect residuals or systematic deviations;
- ask whether the model answers the intended question;
- state which mechanisms are included and which are deliberately lumped;
- communicate uncertainty and limitations.

15 Worked Micro-Examples

15.1 Example 1: a urinary excretion accounting state

Suppose a drug amount q_c is in central compartment and cumulative urinary amount is q_u . Let renal excretion be $k_r q_c$ and nonrenal elimination be $k_n q_c$:

$$\begin{aligned}\dot{q}_c &= u(t) - (k_r + k_n)q_c, \\ \dot{q}_u &= k_r q_c.\end{aligned}$$

The urinary state has no loss because it is cumulative collected amount. If the experiment measures urine over intervals, the observable is not $q_u(t_i)$ itself but increments:

$$\Delta q_{u,i} = q_u(t_i) - q_u(t_{i-1}).$$

This small detail matters for fitting.

15.2 Example 2: a reversible distribution state

For central q_c and tissue q_t :

$$\begin{aligned}\dot{q}_c &= u(t) - k_{ct}q_c + k_{tc}q_t - k_{el}q_c, \\ \dot{q}_t &= k_{ct}q_c - k_{tc}q_t.\end{aligned}$$

Here the tissue state is not cumulative. It can fill and empty. The transfer terms appear with opposite signs. If only central concentration is measured, the tissue state is hidden but may be inferred indirectly from the curve shape.

15.3 Example 3: a simple effect compartment

If plasma concentration C_p changes faster than effect-site concentration C_e , introduce:

$$\dot{C}_e = k_{e0}(C_p - C_e).$$

This equation says the effect site relaxes toward plasma with rate k_{e0} . A direct effect model might use $E(t) = E_{\max}C_p/(EC_{50} + C_p)$. An effect-compartment model uses:

$$E(t) = \frac{E_{\max}C_e(t)}{EC_{50} + C_e(t)}.$$

This is a preview of later *PK/PD*: delays often mean a missing state or a missing turnover process.

16 Summary

16.1 One paragraph

The first three BMEG 671 meetings teach a modeling language rather than a list of equations. A compartment is a deliberate abstraction; a state is the quantity that changes; an input encodes the experimental protocol; an output connects hidden dynamics to measurements. Mass balance turns diagrams into ODEs. Step and impulse inputs distinguish sustained supply from instantaneous dosing. One-compartment models teach exponential decay, steady state, and code verification. Two-compartment models teach matrix form, transfer signs, symmetry, and output maps. Butyrate and SIR examples show that the same grammar scales from *PK* to microbiome-bone biology and infectious disease. Nonlinear elimination shows why saturation changes qualitative behavior. Verification checks whether the equations and code match; validation asks whether the model is adequate for a question. This is the foundation on which the rest of the course will build controllability, observability, identifiability, estimation, uncertainty, and model selection.

16.2 The sentence to keep

For any new biomedical model, ask:

What are the states, what moves between them, what is measured,
and what decision will this model support?

If that sentence is clear, the mathematics has a place to stand.

References

- [1] ASME. V&V 40: Assessing credibility of computational modeling through verification and validation: Application to medical devices, 2018. <https://www.asme.org/codes-standards/find-codes-standards/assessing-credibility-of-computational-modeling-through-verification-and-validation-application-to-medical-devices>.
- [2] Joseph J. DiStefano. *Dynamic Systems Biology Modeling and Simulation*. Academic Press, 2015.
- [3] International Council for Harmonisation. M15: General principles for model-informed drug development, 2026. Step 5 guideline, https://www.ema.europa.eu/en/documents/scientific-guideline/ich-m15-guideline-general-principles-model-informed-drug-development-step-5_en.pdf.
- [4] Mohammad Aminul Islam. BMEG/ELEG 471/671 lecture 1: Compartmental model, 2025. University of Delaware local lecture notes, 27 August 2025.
- [5] Mohammad Aminul Islam. BMEG/ELEG 471/671 lecture 2: Nonlinear compartmental models and matlab, 2025. University of Delaware local lecture notes, 3 September 2025.
- [6] Mohammad Aminul Islam. BMEG/ELEG 471/671 transcript: Compartmental modeling introduction, 2025. University of Delaware local course transcript, 27 August 2025.
- [7] Mohammad Aminul Islam. BMEG/ELEG 471/671 transcript: Impulse, step, verification, and two-compartment models, 2025. University of Delaware local course transcript, 29 August 2025.
- [8] Mohammad Aminul Islam. BMEG/ELEG 471/671 transcript: Two-compartment code, nonlinear elimination, and sir, 2025. University of Delaware local course transcript, 3 September 2025.
- [9] Mohammad Aminul Islam. Nonlinear compartmental, stability, limit cycle, virus-host, and vaccine dosing matlab live scripts, 2025. University of Delaware local MATLAB live scripts.
- [10] Mohammad Aminul Islam. Single-compartment impulse input linear model matlab live script, 2025. University of Delaware local MATLAB live script.
- [11] Mohammad Aminul Islam. Single-compartment step input linear model matlab live script, 2025. University of Delaware local MATLAB live script.
- [12] Mohammad Aminul Islam. Two-compartment step input linear model matlab live script, 2025. University of Delaware local MATLAB live script.
- [13] Mohammad Aminul Islam and Ryan Zurakowski. BMEG/ELEG 471/671: Advanced biomedical and pharmaceutical modeling, 2025. University of Delaware Fall 2025 syllabus, local course file.

- [14] MathWorks. Simbiology tutorials for QSP, PBPK, and PK/PD modeling and analysis, 2026. <https://www.mathworks.com/videos/series/simbiology-tutorials-for-qsp-pbpbk-and-pk-pd-modeling-and-analysis.html>.
- [15] SBML Team. SBML level 3 version 2 core, release 2, 2019. <https://sbml.org/specifications/sbml-level-3/version-2/core/release-2/>.
- [16] Abdul Malik Tyagi, Mingcan Yu, Trevor M. Darby, Chiara Vaccaro, Jau-Yi Li, Joshua A. Owens, Emory Hsu, Jonathan Adams, M. Neale Weitzmann, Rheinallt M. Jones, and Roberto Pacifici. The microbial metabolite butyrate stimulates bone formation via T regulatory cell-mediated regulation of WNT10B expression. *Immunity*, 49(6):1116–1131.e7, 2018.
- [17] U.S. Food and Drug Administration. Model-informed drug development paired meeting program, 2026. <https://www.fda.gov/drugs/development-resources/model-informed-drug-development-paired-meeting-program>.